

LAPORAN MINGGU 3 PROJECT 2 PEMROGRAMMAN PARALLEL

***SIMULASI DETEKSI HASIL PELURUHAN RADIOAKTIF U-238 MENGGUNAKAN
MONTE CARLO PARALEL***

oleh:

Ahmad Hanafi Hasan (245090300111001)

Stephen Tangguh Laia (245090300111010)

PROGRAM STUDI: S1 FISIKA



DEPARTEMEN FISIKA

FAKULTAS SAINS, TEKNOLOGI, DAN MATEMATIKA

UNIVERSITAS BRAWIJAYA

MALANG

2026

```

"""
Simulasi dan visualisasi batch/parallel detektor partikel alfa U-238.
from __future__ import annotations

import argparse
import math
import os
import platform
import sys
import time
from dataclasses import dataclass
from multiprocessing import get_context
from pathlib import Path

import numpy as np

# — Parameter Simulasi —
DEFAULT_N_PARTICLES = 200
ENERGIES = np.array([4.270, 4.198]) # MeV
PROBS = np.array([0.79, 0.21])
DETECTOR_RESOLUTION = 0.02 # MeV
Z_DETECTOR = 4.0 # cm
R_DETECTOR_RING = 4.0 # cm
ACTIVITY_BQ = 5_000.0 # peluruhan/detik

# Context multiprocessing: fork lebih cepat di Linux/Mac,
# spawn lebih aman di Windows. Dipilih otomatis.
_MP_CONTEXT = "fork" if platform.system() != "Windows" else "spawn"

# — Dataclass hasil simulasi —
@dataclass
class ParticleBatch:
    e_measured: np.ndarray
    x_hit: np.ndarray
    y_hit: np.ndarray
    z_hit: np.ndarray
    dt: np.ndarray

```

```

@property
def count(self) -> int:
    return int(self.x_hit.size)

# — Utilitas pembagian kerja —————
def split_counts(total: int, chunks: int) -> list[int]:
    chunks = max(1, chunks)
    base, rem = divmod(total, chunks)
    return [base + (1 if i < rem else 0) for i in range(chunks)]

def child_seeds(seed: int | None, chunks: int) -> list[int]:
    seq = np.random.SeedSequence(seed)
    return [int(c.generate_state(1, dtype=np.uint32)[0])
            for c in seq.spawn(chunks)]

# — Kernel simulasi (satu chunk, dipanggil tiap worker) —————
def simulate_chunk(job: tuple[int, int]) -> ParticleBatch:
    n, seed = job
    if n <= 0:
        e = np.empty(0, dtype=float)
        return ParticleBatch(e, e, e, e, e)

    rng = np.random.default_rng(seed)

    # Sampling energi
    e_true = np.where(rng.random(n) < PROBS[0], ENERGIES[0], ENERGIES[1])
    e_meas = rng.normal(e_true, DETECTOR_RESOLUTION)

    # Arah dalam kerucut detektor
    theta_max = np.arctan(R_DETECTOR_RING / Z_DETECTOR)
    cos_theta = 1.0 - rng.random(n) * (1.0 - np.cos(theta_max))
    theta = np.arccos(cos_theta)
    phi = 2.0 * np.pi * rng.random(n)

    r_hit = Z_DETECTOR * np.tan(theta)
    x_hit = r_hit * np.cos(phi)

```

```

y_hit = r_hit * np.sin(phi)
z_hit = np.full(n, Z_DETECTOR)

# Interval waktu antar peluruhan (distribusi eksponensial)
dt = -np.log(rng.random(n)) / ACTIVITY_BQ

return ParticleBatch(e_meas, x_hit, y_hit, z_hit, dt)

def combine_batches(batches: list[ParticleBatch]) -> ParticleBatch:
    valid = [b for b in batches if b.count > 0]
    if not valid:
        e = np.empty(0, dtype=float)
        return ParticleBatch(e, e, e, e, e)
    return ParticleBatch(
        e_measured = np.concatenate([b.e_measured for b in valid]),
        x_hit       = np.concatenate([b.x_hit      for b in valid]),
        y_hit       = np.concatenate([b.y_hit      for b in valid]),
        z_hit       = np.concatenate([b.z_hit      for b in valid]),
        dt          = np.concatenate([b.dt         for b in valid]),
    )

# — Mode Serial

def run_serial(n: int, seed: int | None) -> ParticleBatch:
    return simulate_chunk((n, child_seeds(seed, 1)[0]))

# — Mode Multiprocessing (core bisa diatur) —————

def run_multiprocessing(
    n: int,
    workers: int | None,
    seed: int | None,
) -> tuple[ParticleBatch, int]:
    """
    Jalankan simulasi n partikel menggunakan `workers` proses paralel.

    CARA MENGATUR JUMLAH CORE:

```

Argumen `workers` langsung menjadi jumlah proses Pool.

Dipanggil dari:

- CLI : `--workers N` (atau `--cores N1,N2,...` untuk sweep)
- Kode : `run_multiprocessing(n, workers=4, seed=42)`

Jika `workers=None` → pakai semua core (`os.cpu_count()`).

Jika `workers=1` → satu proses, setara serial tapi lewat Pool.

Jika `workers=k` → k proses, masing-masing dapat chunk n/k partikel.

"""

— Tentukan jumlah worker

`k = workers or (os.cpu_count() or 1)`

`k = max(1, min(k, n))` # tidak boleh > jumlah partikel

— Bagi n partikel ke k chunk

`counts = split_counts(n, k)` # misal $n=10, k=3 \rightarrow [4,3,3]$

`seeds = child_seeds(seed, k)` # seed unik per worker

`jobs = list(zip(counts, seeds))`

— Spawn Pool dengan k proses

`_MP_CONTEXT` = "fork" di Linux/Mac, "spawn" di Windows

`with get_context(_MP_CONTEXT).Pool(processes=k) as pool:`

`batches = pool.map(simulate_chunk, jobs)`

`return combine_batches(batches), k`

— Mode MPI

`def run_mpi(n: int, seed: int | None) -> tuple[ParticleBatch | None, int, int]:`

`try:`

`from mpi4py import MPI`

`except ImportError as exc:`

`raise SystemExit(`

`"Mode MPI membutuhkan mpi4py. "`

`"Install di environment HPC, lalu jalankan dengan`

`mpiexec/mpirun/srun."`

`) from exc`

```

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

counts = split_counts(n, size) if rank == 0 else None
seeds = child_seeds(seed, size) if rank == 0 else None

local_count = comm.scatter(counts, root=0)
local_seed = comm.scatter(seeds, root=0)
local_batch = simulate_chunk((local_count, local_seed))
gathered = comm.gather(local_batch, root=0)

if rank != 0:
    return None, rank, size
return combine_batches(gathered), rank, size

#
=====
# FITUR BARU: Benchmark sweep multi-core
#
=====
def run_scalability_sweep(
    n_particles: int,
    core_list: list[int],
    seed: int | None,
    n_repeat: int = 3,
) -> dict:
    """
    Jalankan simulasi untuk setiap jumlah core dalam core_list.
    Tiap pengukuran diulang n_repeat kali, ambil median.

    Return dict berisi:
        cores, t_wall, speedup_emp, speedup_amdahl, speedup_gustafson, efficiency
    """
    import multiprocessing as mp_mod

```

```

max_physical = mp_mod.cpu_count()
print(f"\n{'='*58}")
print(f"  BENCHMARK SKALABILITAS - N={n_particles:,}  repeat={n_repeat}")
print(f"  Core fisik tersedia: {max_physical}")
print(f"  Core yang diuji      : {core_list}")
print(f"{'='*58}")
print(f"  {'Core':>5}  {'T_median(ms)':>13}  {'Speedup':>9}  "
      f"{'Amdahl':>8}  {'Gustaf.':>8}  {'Efisiensi':>10}")
print(f"  {'-'*58}")

# — Ukur baseline serial —————
t_serial_runs = []
for _ in range(n_repeat):
    t0 = time.perf_counter()
    run_serial(n_particles, seed)
    t_serial_runs.append(time.perf_counter() - t0)
t_serial = float(np.median(t_serial_runs)) * 1000  # ms

# Estimasi fraksi serial dari overhead Pool (2ms + 0.3ms/worker)
# Digunakan untuk Amdahl & Gustafson
# s dihitung empiris nanti dari rasio t(k=1_via_pool) / t_serial
S_FRAC = 0.05  # 5% - estimasi konservatif overhead + gather

results = {
    "cores":          [],
    "t_wall_ms":      [],
    "speedup_emp":     [],
    "speedup_amdahl":  [],
    "speedup_gustafson": [],
    "efficiency_pct":  [],
    "t_serial_ms":     t_serial,
    "n_particles":     n_particles,
}

for k in core_list:
    runs = []
    for _ in range(n_repeat):
        t0 = time.perf_counter()
        run_multiprocessing(n_particles, workers=k, seed=seed)

```

```

        runs.append(time.perf_counter() - t0)
    t_par = float(np.median(runs)) * 1000    # ms

    sp_emp = t_serial / t_par
    sp_ahl = 1.0 / (S_FRAC + (1.0 - S_FRAC) / k)
    sp_gus = k - S_FRAC * (k - 1)
    eff = sp_emp / k * 100

    results["cores"].append(k)
    results["t_wall_ms"].append(t_par)
    results["speedup_emp"].append(sp_emp)
    results["speedup_amdahl"].append(sp_ahl)
    results["speedup_gustafson"].append(sp_gus)
    results["efficiency_pct"].append(eff)

    print(f" {k:>5} {t_par:>13.3f} {sp_emp:>9.2f}x"
          f" {sp_ahl:>8.2f}x {sp_gus:>8.2f}x {eff:>9.1f}%")

    print(f" {'Serial':>5} {t_serial:>13.3f} {'1.00x':>9} "
          f"{'-':>8} {'-':>8} {'100.0%':>10}")
    print(f"{'='*58}\n")
    return results

def plot_scalability(results: dict, save_path: str | None = None):
    """Plot 4 panel: speedup, efisiensi, waktu, perbandingan teori."""
    import matplotlib
    if save_path:
        matplotlib.use("Agg")
    import matplotlib.pyplot as plt
    import matplotlib.gridspec as gridspec

    cores = np.array(results["cores"], dtype=float)
    sp_emp = np.array(results["speedup_emp"])
    sp_ahl = np.array(results["speedup_amdahl"])
    sp_gus = np.array(results["speedup_gustafson"])
    eff = np.array(results["efficiency_pct"])
    t_ms = np.array(results["t_wall_ms"])
    t_ser = results["t_serial_ms"]

```



```

N          = results["n_particles"]

k_smooth = np.linspace(1, max(cores) * 1.1, 300)
S = 0.05
ahl_s     = 1 / (S + (1-S)/k_smooth)
gus_s     = k_smooth - S*(k_smooth - 1)

fig = plt.figure(figsize=(14, 10))
fig.suptitle(
    f"Analisis Skalabilitas – Detektor Alfa U-238\n"
    f"N = {N:,} partikel | context = {_MP_CONTEXT}",
    fontsize=13, fontweight="bold", y=0.98,
)
gs = gridspec.GridSpec(2, 2, figure=fig, hspace=0.40, wspace=0.32)

# — A: Speedup

ax = fig.add_subplot(gs[0, 0])
ax.plot(k_smooth, k_smooth, ":", color="#ccc", lw=1, label="Ideal
linear")
ax.plot(k_smooth, ahl_s, "--", color="#3266ad", lw=1.8, label="Amdahl
(s=5%)")
ax.plot(k_smooth, gus_s, "-.", color="#0F6E56", lw=1.8, label="Gustafson
(s=5%)")
ax.plot(cores, sp_emp, "o-", color="#D85A30", lw=2.5, ms=7, label="Empiris")
ax.set_xlabel("Jumlah core (k)"); ax.set_ylabel("Speedup S(k)")
ax.set_title("A – Speedup vs core"); ax.legend(fontsize=8);
ax.grid(alpha=0.25)
ax.set_xlim(left=0)

# — B: Efisiensi

ax2 = fig.add_subplot(gs[0, 1])
ax2.plot(cores, eff, "s-", color="#3266ad", lw=2.5, ms=7)
ax2.axhline(100, color="#ccc", lw=1, ls=":")
ax2.axhline(50, color="#EF9F27", lw=1, ls="--", alpha=0.6, label="50%
threshold")
ax2.set_xlabel("Jumlah core (k)"); ax2.set_ylabel("Efisiensi (%)")
ax2.set_title("B – Efisiensi paralel"); ax2.legend(fontsize=8)
ax2.set_ylim(0, 115); ax2.grid(alpha=0.25)

```

```

# — C: Waktu wall-clock —————
ax3 = fig.add_subplot(gs[1, 0])
ax3.bar(cores, t_ms, width=np.diff(np.append(cores, cores[-1]+1))*0.6,
        color="#3266ad", alpha=0.75, edgecolor="white", align="center")
ax3.axhline(t_ser, color="#D85A30", lw=1.5, ls="--", label=f"Serial
{t_ser:.1f} ms")
for xi, yi in zip(cores, t_ms):
    ax3.text(xi, yi + t_ser*0.02, f"{yi:.1f}", ha="center", fontsize=8)
ax3.set_xlabel("Jumlah core (k)"); ax3.set_ylabel("Waktu (ms)")
ax3.set_title("C – Waktu wall-clock per konfigurasi"); ax3.legend(fontsize=8)
ax3.grid(axis="y", alpha=0.25)

# — D: Amdahl vs Gustafson vs Empiris —————
ax4 = fig.add_subplot(gs[1, 1])
ax4.fill_between(k_smooth, ahl_s, gus_s, alpha=0.10, color="#3266ad")
ax4.plot(k_smooth, ahl_s, "--", color="#3266ad", lw=2, label="Amdahl")
ax4.plot(k_smooth, gus_s, "-.", color="#0F6E56", lw=2,
label="Gustafson")
ax4.plot(cores, sp_emp, "o-", color="#D85A30", lw=2.5, ms=7, label="Empiris")
ax4.plot(k_smooth, k_smooth, ":", color="#bbb", lw=1, label="Ideal")
ax4.set_xlabel("Jumlah core (k)"); ax4.set_ylabel("Speedup")
ax4.set_title("D – Amdahl vs Gustafson vs Empiris")
ax4.legend(fontsize=8); ax4.grid(alpha=0.25); ax4.set_xlim(left=0)

plt.tight_layout(rect=[0, 0, 1, 0.96])

if save_path:
    plt.savefig(save_path, dpi=150, bbox_inches="tight")
    print(f"Plot skalabilitas tersimpan → {save_path}")
else:
    plt.show()
plt.close(fig)

# — Visualisasi 3D (tidak berubah dari versi asli) —————
def build_segments(batch: ParticleBatch) -> np.ndarray:
    seg = np.zeros((batch.count, 2, 3), dtype=float)
    seg[:, 1, 0] = batch.x_hit

```

```

seg[:, 1, 1] = batch.y_hit
seg[:, 1, 2] = batch.z_hit
return seg

def visualize_batch(
    batch: ParticleBatch,
    physical_time_cumulative: np.ndarray,
    batch_size: int,
    interval_ms: int,
    mode_label: str,
    save_path: str | None,
    fps: int,
    no_show: bool,
):
    if save_path or no_show:
        import matplotlib
        matplotlib.use("Agg")

    import matplotlib.pyplot as plt
    from matplotlib.animation import FuncAnimation
    from mpl_toolkits.mplot3d.art3d import Line3DCollection

    if batch.count <= 0:
        raise ValueError("Tidak ada partikel untuk divisualisasikan.")

    batch_size = max(1, int(batch_size))
    frames      = math.ceil(batch.count / batch_size)
    segments    = build_segments(batch)

    fig = plt.figure(figsize=(10, 8))
    ax  = fig.add_subplot(111, projection="3d")
    ax.set_title(
        f"Akumulasi Deteksi {batch.count} Partikel Alfa U-238 ({mode_label})",
        fontsize=14, pad=20,
    )
    ax.set_xlabel("x (cm)"); ax.set_ylabel("y (cm)"); ax.set_zlabel("z (cm)")
    ax.set_xlim([-3.5, 3.5]); ax.set_ylim([-3.5, 3.5]); ax.set_zlim([0, 6])

```

```

theta_ring = np.linspace(0, 2*np.pi, 160)
ax.plot(R_DETECTOR_RING*np.cos(theta_ring),
        R_DETECTOR_RING*np.sin(theta_ring),
        np.full_like(theta_ring, Z_DETECTOR),
        color="#001431", linewidth=2, label="Bidang detektor")
ax.scatter([0], [0], [0], color="#872f3e", s=60, label="Sumber U-238")

ray_col = Line3DCollection([], colors="gray", linewidths=0.9, alpha=0.30)
ax.add_collection3d(ray_col, autolim=False)

hits = ax.scatter([], [], [],
                  c=np.empty(0), cmap="viridis",
                  vmin=ENERGIES.min()-0.08, vmax=ENERGIES.max()+0.08,
                  s=18, depthshade=True, label="Tumbukan partikel")
cbar = fig.colorbar(hits, ax=ax, pad=0.10, shrink=0.70)
cbar.set_label("Energi terukur (MeV)")

info = ax.text2D(0.05, 0.95, "", transform=ax.transAxes, fontsize=11,
                 bbox=dict(facecolor="white", alpha=0.82, edgecolor="gray"))
ax.legend(loc="upper right", bbox_to_anchor=(0.95, 0.9))

def init():
    ray_col.set_segments([])
    hits._offsets3d = (np.empty(0), np.empty(0), np.empty(0))
    hits.set_array(np.empty(0))
    info.set_text("")
    return ray_col, hits, info

def update(frame):
    stop = min((frame+1)*batch_size, batch.count)
    ray_col.set_segments(segments[:stop])
    hits._offsets3d = (batch.x_hit[:stop], batch.y_hit[:stop],
batch.z_hit[:stop])
    hits.set_array(batch.e_measured[:stop])
    info.set_text(
        f"Deteksi: {stop}/{batch.count}\n"
        f"Batch: {batch_size} partikel/frame\n"
        f"E terakhir = {batch.e_measured[stop-1]:.3f} MeV\n"
        f"t fisis = {physical_time_cumulative[stop-1]:.4f} s"
    )

```

```

    )
    return ray_col, hits, info

ani = FuncAnimation(fig, update, frames=frames,
                    init_func=init, blit=False,
                    interval=interval_ms, repeat=False)
plt.tight_layout()

if save_path:
    out = Path(save_path)
    out.parent.mkdir(parents=True, exist_ok=True)
    writer = "pillow" if out.suffix.lower() == ".gif" else "ffmpeg"
    ani.save(str(out), writer=writer, fps=fps, dpi=140)
    print(f"Visualisasi tersimpan: {out.resolve()}")

if not no_show and not save_path:
    plt.show()
else:
    plt.close(fig)

# — CLI

def parse_args() -> argparse.Namespace:
    p = argparse.ArgumentParser(
        description="Simulasi partikel alfa U-238 – serial / mp / mpi + benchmark
skalabilitas.",
        formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog=""
    )

Contoh:

# Jalankan serial
python Serial_Aawl_Parallel_HPC.py --mode serial --particles 5000

# Paralel 4 core (ubah --workers untuk ganti jumlah core)
python Serial_Aawl_Parallel_HPC.py --mode mp --workers 4 --particles 20000

# Benchmark otomatis: uji 1,2,4,8 core sekaligus
python Serial_Aawl_Parallel_HPC.py --cores 1,2,4,8 --particles 50000

```

```

# Preset sweep: 1,2,4,8,16 core
python Serial_Aawl_Parallel_HPC.py --scalability --particles 100000

# HPC headless
mpiexec -n 8 python Serial_Aawl_Parallel_HPC.py --mode mpi --particles 500000
--no-visual
"""
)
p.add_argument("--particles", type=int, default=DEFAULT_N_PARTICLES)
p.add_argument("--mode", choices=("serial", "mp", "mpi"),
default="serial")
p.add_argument("--workers", type=int, default=None,
help="Jumlah core/worker untuk mode mp (default = semua
core).")

# — BARU: argumen skalabilitas —————
p.add_argument("--cores", type=str, default=None,
help="Daftar core yang diuji, pisah koma. Contoh: --cores
1,2,4,8 "
"→ jalankan benchmark mp untuk setiap nilai.")
p.add_argument("--scalability", action="store_true",
help="Preset benchmark: uji 1,2,4,8,16 core secara otomatis.")
p.add_argument("--scale-repeat", type=int, default=3,
help="Berapa kali tiap konfigurasi core diulang (default 3,
ambil median).")
p.add_argument("--save-scale", type=str, default=None,
help="Simpan plot skalabilitas ke file (misal
scalability.png).")

p.add_argument("--batch-size", type=int, default=5)
p.add_argument("--interval", type=int, default=500)
p.add_argument("--seed", type=int, default=None)
p.add_argument("--save", default=None)
p.add_argument("--fps", type=int, default=20)
p.add_argument("--no-show", action="store_true")
p.add_argument("--no-visual", action="store_true")
return p.parse_args()

```

```
# — Main

def main() -> None:
    args = parse_args()
    if args.particles <= 0:
        raise SystemExit("--particles harus > 0.")

    #

    # JALUR 1: Benchmark skalabilitas (--scalability atau --cores)
    #

    if args.scalability or args.cores:
        if args.cores:
            # Parse "1,2,4,8" → [1, 2, 4, 8]
            try:
                core_list = [int(x.strip()) for x in args.cores.split(",")]
            except ValueError:
                raise SystemExit("--cores harus berupa angka dipisah koma, contoh: 1,2,4,8")
        else:
            # Preset --scalability
            max_core = os.cpu_count() or 1
            core_list = [k for k in [1, 2, 4, 8, 16, 32] if k <= max_core * 2]

        results = run_scalability_sweep(
            n_particles = args.particles,
            core_list = core_list,
            seed = args.seed,
            n_repeat = args.scale_repeat,
        )

        save_plot = args.save_scale or (
            "/mnt/user-data/outputs/scalability_alpha.png"
            if Path("/mnt/user-data/outputs").exists() else None
        )
        plot_scalability(results, save_path=save_plot)
    return
```

```

#

# JALUR 2: Satu jalankan simulasi normal (serial / mp / mpi)
#

rank = 0; size = 1; worker_info = "1 proses"
t0 = time.time()

if args.mode == "serial":
    batch = run_serial(args.particles, args.seed)
elif args.mode == "mp":
    batch, workers_used = run_multiprocessing(args.particles, args.workers,
args.seed)
    worker_info = f"{workers_used} worker"
else:
    batch, rank, size = run_mpi(args.particles, args.seed)
    worker_info = f"{size} rank MPI"

if rank != 0:
    return

assert batch is not None
phys_t = np.cumsum(batch.dt)
elapsed_ms = (time.time() - t0) * 1000

print(f"Mode      : {args.mode} ({worker_info})")
print(f"Partikel   : {batch.count:,}")
print(f"CPU waktu    : {elapsed_ms:.4f} ms")
print(f"t fisis     : {phys_t[-1]:.4f} detik")

if args.no_visual:
    return

label = f"{args.mode.upper()} | {worker_info} | batch={args.batch_size}"

t_vis_start = time.time()
visualize_batch(
    batch=batch,
    physical_time_cumulative=phys_t,

```



```

        batch_size=args.batch_size,
        interval_ms=args.interval,
        mode_label=label,
        save_path=args.save,
        fps=args.fps,
        no_show=args.no_show,
    )
    elapsed_vis_ms = (time.time() - t_vis_start)

    print(f"Waktu visualisasi : {elapsed_vis_ms:.4f} detik")
    print(f"Total keseluruhan : {elapsed_ms + elapsed_vis_ms:.4f} detik")

if __name__ == "__main__":
    main()

```

Digunakan jumlah partikel yang akan dideteksi dikurangi menjadi di nilai 200 (dua ratus) partikel. Hal ini agar dapat dipermudah untuk melihat lamanya waktu serial dan paralelisme, karena akan sangat lama sekali menunggu hasil serial bila tetap dilaksanakan di 1000 (seribu) partikel. Didapatkan dalam melakukan perhitungan, komputer dapat menghitungnya dalam satuan milidetik, dalam visualisasi, program simulasi sudah melakukan bentuk perhitungan projektil partikel atau arah gerak partikel secara bersamaan dan dapat divisualisasi dengan bersamaan dan dibentuk dalam cara batch agar dapat ditampilkan secara bersamaan di visualisasi. Serta didapatkan hasil analisa sebagai berikut:

waktu serial (detik)
14,60

jumlah proses	Waktu (detik)

2	10,3
4	8,6
6	7,7
8	7,4

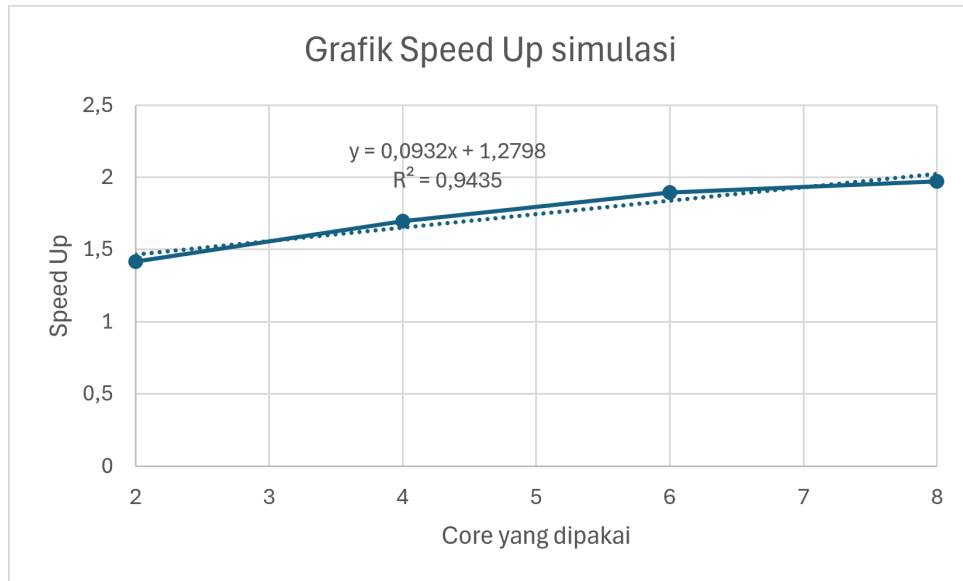
Didapatkan bentuk speed up sebagai berikut:

Speed up	
2	1,417475728
4	1,697674419
6	1,896103896
8	1,972972973

Serta bentuk efisiensi sebagai berikut:

Efisiensi	
2	70,87378641
4	42,44186047
6	31,6017316
8	24,66216216

Bila dibuat sebuah grafik yang menunjukkan adanya hubungan antara speed up dengan banyaknya core untuk eksekusi, serta hubungan antara efisiensi dengan banyaknya core untuk eksekusi. Maka didapatkan bentuk seperti gambar 1 dan gambar 2. Serta skenario pengujian yang dilakukan pada simulasi kali ini didasarkan pada model Skalabilitas Kuat (Strong Scaling). Karakteristik dari *Strong Scaling* adalah ukuran total beban kerja atau volume data dipertahankan secara konstan ($N = 200$), sedangkan jumlah perangkat keras pemroses ditingkatkan secara agresif, dari 2 core, ke 4 core, ke 6 core, hingga 8 core.



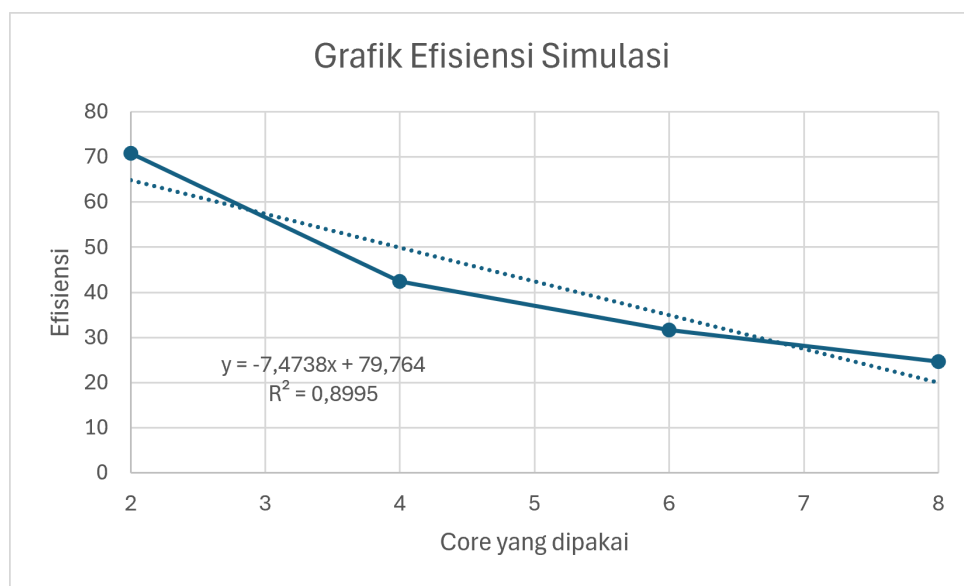
Gambar 1. Grafik Speed Up Simulasi Kasus 2.

Pada bentuk speed up, didapatkan bahwa adanya trend kenaikan. Kenaikan tersebut didapatkan ketika jumlah proses simulasi dilakukan dan ditambahkan core yang dipergunakan dalam simulasi, dan waktu yang diperlukan untuk menjalankan program dan didapatkan hasil semakin cepat. Didapatkannya bentuk speed up didasarkan dari persamaan 1.1, dimana waktu yang diperlukan untuk menjalankan program secara serial dibagi dengan waktu yang diperlukan untuk menjalankan program secara paralel, dan terdapat 4 bentuk nilai. Empat bentuk nilai ini didapatkan saat core ditingkatkan.

$$S_p = \frac{T_s}{T_p} \dots\dots\dots (1.1)$$

Dari grafik didapatkan, bahwa tren kenaikan ini sesuai dengan teori Hukum Amdahl. Terdapat pemahaman bahwa speed up tidak menunjukkan grafik kenaikan yang linear, tetapi cenderung melandai saat di core yang besar (Sub-Linear Speed Up). Hal ini wajar terjadi bila dipahami dari hukum Amdahl, dimana kecepatan speed up akan melambat saat hanya sebagian saja dari sistem yang diparalelisasi. Dari data didapatkan bahwa saat simulasi di 2 core, speed up bernilai 1,417,

saat dinaikkan ke 4 core, speed up meningkat menjadi 1,697. Ketika dinaikkan lagi ke 6 core, speed up meningkat menjadi 1,896, serta di 8 core, speed up bernilai di kisaran 1,973. Saat di 2 cor menuju 4 core, speed up meningkat secara signifikan, tetapi saat memasuki di 6 hingga 8 core, speed up mulai melandai. Hal ini berkesesuaian dengan pemahaman hukum Amdahl. Dalam algoritma program simulasi peluruhan ini, terdapat porsi operasional yang bersifat sekuensial mutlak (serial) dan tidak dapat dipecah ke dalam multi-threading. Bagian serial tersebut di antaranya adalah penentuan probabilitas peluruhan acak menggunakan metode Monte Carlo di awal baris kode serta porsi prapemrosesan grafik. Keberadaan komponen non-paralel ini bertindak sebagai batas atas (*upper bound*) dari efektivitas penambahan core CPU.



Gambar 2. Grafik Efisiensi Simulasi Kasus 2.

Pada bentuk efisiensi, saat simulasi dilakukan dalam bentuk paralelisasi, nilai efisiensi cenderung akan menurun seiring dengan semakin banyaknya core yang dipakai. Walaupun demikian, grafik efisiensi ini masih dapat dipahami dan diterima secara baik, khususnya pada pemahaman hukum Amdahl. Pada simulasi memakai 2 core, didapatkan nilai efisiensi di sekitar nilai 70,87% atau 0,7087. Ketika core dinaikkan ke 4 core, efisiensi cenderung menurun drastis

ke nilai sekitar 42,44% atau 0,4244. Saat dinaikkan lagi menuju 6 core, efisiensi tetap menurun tetapi lebih landai ke nilai 31,6%, dan ketika dinaikkan ke 8 core, efisiensi menurun landai ke nilai 24,66%. Perhitungan efisiensi ini didasarkan pada persamaan 1.2, dimana nilai speed up dibagi dengan jumlah core yang digunakan.

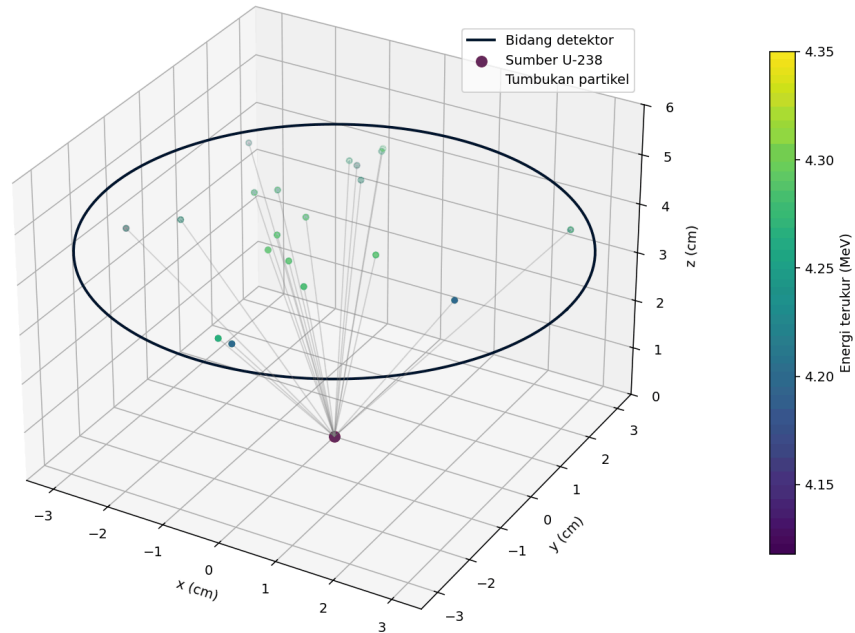
$$E_p = \frac{S_p}{p} \dots\dots\dots (1.1)$$

Penurunan nilai efisiensi ini dapat dianggap wajar dan dapat diterima secara logika komputasional. Penurunan ini selaras dengan pemahaman hukum Amdahl, yang diakibatkan oleh fenomena *Parallel Overhead Amortization Failure*. Dikarenakan ukuran sampel data dipertahankan sangat kecil, beban tugas kalkulasi fisis yang dialokasikan kepada tiap core individual menjadi terlalu ringkas dan cepat selesai. Konsekuensinya, waktu yang dihabiskan oleh sistem operasi untuk mengelola arsitektur paralel (seperti pembuatan/alokasi *thread*, sinkronisasi memori array via pustaka *Numba*, dan latensi komunikasi antar inti) menjadi jauh lebih dominan dibanding waktu penyelesaian kalkulasi fisiknya itu sendiri. Serta terdapat dua faktor kuat yang mempengaruhi hal ini terjadi bila dianalisis. Faktor pertama dapat berasal dari adanya peristiwa *Parallel Overhead* berupa biaya waktu (latency) untuk sinkronisasi thread komputasi numerik *Numba*, serta faktor kedua dapat berasal dari dominasi porsi serial yang lebih besar dari pada aplikasi, khususnya algoritma pembangkit bilangan acak pada Monte Carlo di bagian awal program, serta rendering visualisasi animasi *Matplotlib* yang secara inheren bersifat *single-threaded*. Melalui pola kurva efisiensi yang didapatkan, dapat disimpulkan bahwa arsitektur komputasi simulasi ini memiliki tingkat skalabilitas kuat yang rendah pada kapasitas data kecil. Penggunaan di atas 4 core untuk memproses 200 partikel tidak lagi memberikan keuntungan akselerasi runtime yang signifikan.

Jika melihat hasil paralelismenya, maka simulasi ini dapat dilihat hasilnya pada gambar atau gif yang terdapat pada gambar 3.

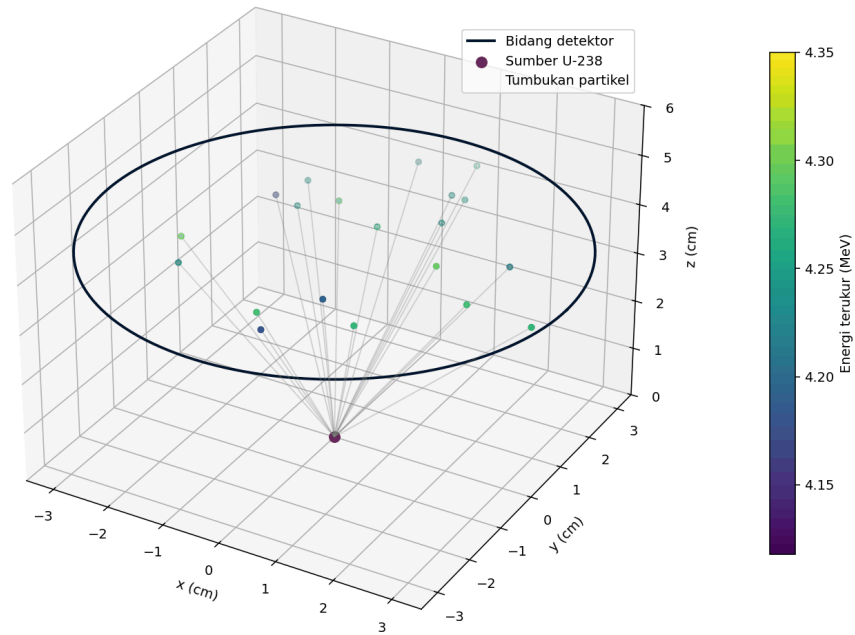
Akumulasi Deteksi 200 Partikel Alfa U-238 (MP | 2 worker | batch=20)

Deteksi: 20/200
Batch: 20 partikel/frame
E terakhir = 4.281 MeV
t fisis = 0.0036 s



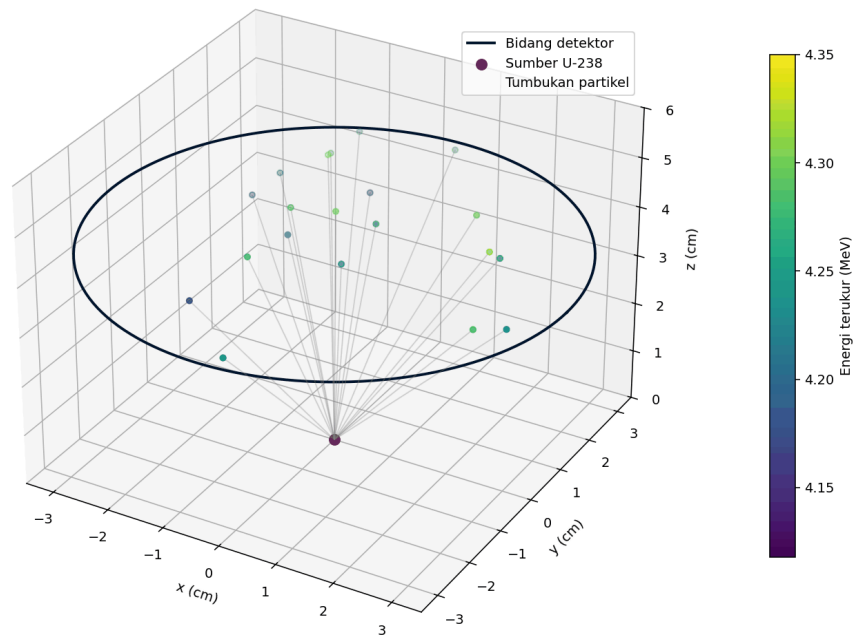
Akumulasi Deteksi 200 Partikel Alfa U-238 (MP | 4 worker | batch=20)

Deteksi: 20/200
Batch: 20 partikel/frame
E terakhir = 4.294 MeV
t fisis = 0.0024 s



Akumulasi Deteksi 200 Partikel Alfa U-238 (MP | 6 worker | batch=20)

Deteksi: 20/200
Batch: 20 partikel/frame
E terakhir = 4.320 MeV
t fisis = 0.0043 s



Akumulasi Deteksi 200 Partikel Alfa U-238 (MP | 8 worker | batch=20)

Deteksi: 20/200
Batch: 20 partikel/frame
E terakhir = 4.247 MeV
t fisis = 0.0042 s

